

POLITECNICO
MILANO 1863

Ingegneria del Software

Ingegneria Informatica – Prof. San Pietro

Esercitazione:

Introduzione Java

My Contacts:
Giuseppe Sorrentino
giuseppe.sorrentino@polimi.it

Svolto A Lezioni

Esercizio 0

- Indicare il comportamento del seguente programma nello heap:

```
public static void main(String[] args) {
    String s1 = "abc";
    String s2 = "abc";
    String s3 = new String("abc");
    String s4 = s3;
}
```

Svolto A Lezioni

Esercizio 0 (soluzione)

- Indicare il comportamento del seguente programma nello heap:

```
public static void main(String[] args) {
    String s1 = "abc";
    String s2 = "abc";
    String s3 = new String("abc");
    String s4 = s3;
}
```

Svolto A Lezioni

Esercizio 0 (soluzione)

- Indicare il comportamento del seguente programma nello heap:

```
public static void main(String[] args) {
    String s1 = "abc";
    String s2 = "abc";
    String s3 = new String("abc");
    String s4 = s3;
}
```

Puntano allo stesso oggetto immutabile nello heap

Svolto A Lezioni

Esercizio 0 (soluzione)

- Indicare il comportamento del seguente programma nello heap:

```
public static void main(String[] args) {
    String s1 = "abc";
    String s2 = "abc";
    String s3 = new String("abc");
    String s4 = s3;
}
```

s3 punta ad un oggetto diverso da s1 e s2
ma è lo stesso oggetto puntato da s4

Esercizio Extra 1:

Un oggetto della classe *AgendaTelefonica* permette di gestire una semplice agenda (rubrica) telefonica. Ogni utenza dell'agenda ha un nome, un cognome e un numero di telefono. L'agenda può gestire un numero limitato di utenze. Tale numero deve essere scelto all'atto della creazione di un oggetto della classe AgendaTelefonica. La classe ha il seguente scheletro:

Esercizio Extra 1:

```
public class AgendaTelefonica{
// nome[i], cognome[i] e numTel[i] memorizzano
// nome, cognome e telefono della i-esima utenza
private Utenza[] utenti;
public AgendaTelefonica(int dim)
/* se l'agenda non è piena, inserisce in agenda
 una nuova utenza con nome n, cognome c
 e numero di telefono t; poi restituisce true.
 Se l'agenda è piena restituisce false
*/
public boolean inserisci(String n, String c, String t)
/* restituisce il numero di telefono di un'utenza
 con nome n e cognome c, se una tale utenza è
 presente in agenda; altrimenti restituisce null
*/
public String trova (String n, String c)
/* visualizza sullo standard output la lista completa
 delle utenze in agenda
*/
public void listaUtenze()
}
```

POLITECNICO MILANO 1863

Esercizio Extra 1:

Scrivere il corpo dei metodi della classe Agenda. Scrivere inoltre una classe di test che:

A) Permette all'utente di creare un'agenda di dimensione a sua scelta;

B) Devono essere possibili le seguenti azioni:

1. Inserimento di una nuova utenza nell'agenda;
2. Ricerca di una nuova utenza nell'agenda;
3. Visualizzazione di tutte le utenze dell'agenda sullo standard output;

Ref: Fondamenti di Informatica in Java, Emilio Di Giacomo, Walter Didimo, Maggioli Ed., 2014.

Esercizio 1: Integer Set

Svolto A Lezioni

- Definire una **classe** che rappresenti un Set di interi
    - Collection of objects
    - Non-ordered
    - Senza ripetizioni **: ogni elemento deve essere controllato prima dell'inserimento**

Esercizio 1: Integer Set

Svolto A Lezioni

- Definire una **classe** che rappresenti un Set di interi
    - Collection of objects
    - Non-ordered
    - Senza ripetizioni **: ogni elemento deve essere controllato prima dell'inserimento**
- **Attributi** della Classe:
    - elements: Array containing the set elements, of a pre-allocated size
    - size: Number of elements actually present in the set

Esercizio 1: Integer Set

Svolto A Lezioni

- Definire una **classe** che rappresenti un Set di interi
    - Collection of objects
    - Non-ordered
    - Senza ripetizioni **: ogni elemento deve essere controllato prima dell'inserimento**
- **Attributi** della Classe:
    - elements: Array containing the set elements, of a pre-allocated size
    - size: Number of elements actually present in the set
- **Metodi** della Classe:
    - Costruttore
    - Check: per vedere se un elemento è già presente nel set
    - Insert/Remove: per inserire e rimuovere gli elementi

Esercizio 2: Counter

Svolto A Lezioni

Creare una classe che tenga traccia del numero di istanze **create**.

Example:

MyCounter = 1
TotCounter = 1

Esercizio 2: Counter

Svolto A Lezioni

Creare una classe che tenga traccia del numero di istanze **create**.

Example:

MyCounter = 1
TotCounter = 2

MyCounter = 2
TotCounter = 2

Esercizio 2: Counter

Svolto A Lezioni

Creare una classe che tenga traccia del numero di istanze **create**.

Example:

MyCounter = 1 TotCounter = 3	MyCounter = 2 TotCounter = 3	MyCounter = 3 TotCounter = 3

Svolto A Lezioni

Esercizio 2: Counter

Creare una classe che tenga traccia del numero di istanze **create**.

Example:

MyCounter = 1 TotCounter = 4	MyCounter = 2 TotCounter = 4	MyCounter = 3 TotCounter = 4	MyCounter = 4 TotCounter = 4

Svolto A Lezioni

Esercizio 2: Counter

Creare una classe che tenga traccia del numero di istanze **create**.

Example:

MyCounter = 1 TotCounter = 4	MyCounter = 2 TotCounter = 4	MyCounter = 3 TotCounter = 4	MyCounter = 4 TotCounter = 4

TotCounter deve essere **static**

Esercizio 3: Rock-Paper-Scissors

- Implementare rock-paper-scissors
- Human player vs COM
- GameLogic: gestisce l'interazione con l'utente
- Choice
  - Enum : gestisce le opzioni (rock/paper/scissors)
- Outcome
  - Enum**:** gestisce i risultati (victory/loss/draw)